Emanuel Islas Quintos

Comparación de métodos gauss jordan y gauss seidel aplicados a un problema de contexto

Algoritmo de Gauss Jordan (imagen generada con IA) ***2***

Algoritmo de Gauss Seidel (imagen generada con IA) ***3***

Problema de contexto que soluciona este código ***4***

Código en Python ***5***

Resultados obtenidos ***7***

GAUSS JORDAN **7**

GAUSS SEIDEL **7**

Solución del sistema utilizando symbolab ***7***

Funcionalidad del código ***8***

Limitaciones del código actual ***8***

Anexos ***8***

Roles y responsabilidades **8**

Grafica tipo Gantt **9**

Algoritmo de Gauss Jordan (imagen generada con IA)

MÉTODO DE GAUSS-JORDAN

(para resolver $Ax = b \longrightarrow$ forma escalonada reducida)

1 **Definir** la matriz de coeficientes A $(n \times n) \longrightarrow$ debe ser cuadrada

2 **Definir** el vector de términos independientes b $(n \times 1)$

3 **Construir** la matriz aumentada $[A \mid b]$ de tamaño $n \times (n+1)$

4 **(Opcional pero recomendado) Verificar** si la matriz A **es invertible**

$\longrightarrow$ Determinante $\neq 0$ o $rango(A) = n$

$\longrightarrow$ Si no $\rightarrow$ el sistema puede no **tener *solución única***

5 **Inicializar:** $k - 1 \longrightarrow$ columna / pivote actual (comienza en 1)

6 **Mientras** $k \leq n$ hacer:

6.1 **Encontrar** pivote en la columna k (desde fila k hasta n)

Buscar la fila $p \geq k$ tal que $[a_{pni,}]$ sea *máximo (pivoteo parcial)*

Si el pivote *máximo* es $0 \longrightarrow$ la columna es **linealmente dependiente**

$\longrightarrow$ el sistema ***no*** tiene solución única o tiene **infinitas soluciones**. Terminar con mensaje de **"*no solución única*"**

6 Si $p \neq k \longrightarrow$ **Intercambiar fila** k con fila p (swap)

6 **6.3** **Normalizar la fila** pivote (hacer el pivote = 1)

factor $= a_{kk}$

Para $j = k$ hasta n+1 hacer:

$a_{kj} = a_{kj} / factor$

6 **6.4** **Eliminar** (hacer **ceros**) en TODA la columna **k, excepto** en la posición pivote

Para cada fila $i = 1$ **hasta** n, $i \neq k$ hacer:

factor $= a_{ik}$ ← *se puede empezaar desde t, pero optinizado desde k*

Para $j = k$ hasta n+1 hacer:

$a_{kj} = a_{ij} - factor * a_{kj}$

6 **6.5** $k \leftarrow k + 1$

7. Al terminar el ciclo ($k > n$):

Si la matriz aumentada está en forma **escalonada reducida** (diagonal principal = 1, resto o

$\longrightarrow$ La solución del sistema está en la última columna:

$x_i = [aumentada]_{1,n+1}$

$x_n = [aumentada]_{2,n+1}$

$$\begin{bmatrix} x_i = [aumentada]_{1,n+1} \\ x_n = [aumentada]_{2,n+1} \end{bmatrix}$$

$\longrightarrow$ Sino (aparecieron filas tipo $(0 \mid 0 \ldots 0 \mid \epsilon \mid$ con $c \neq 0)$:

$\longrightarrow$ Sistema inconsistente: $\longrightarrow$ No tiene solución

$\longrightarrow$ Sino (hay filas $[0 \mid 0 \ldots 0 \mid 0]$ y **variables libres:**

8. $\longrightarrow$ **Infinitas soluciones** (sistema subdeterminado)

Algoritmo de Gauss Seidel (imagen generada con IA)

MÉTODO DE GAUSS-SEIDEL

(para resolver $Ax = b$)

1 **Definir** la matriz de coeficientes A $(n \times n)$
→ Debe ser cuadrada

2 **Definir** el vector de términos independientes b $(n \times 1)$

3 **Definir** el vector de aproximación inicial $X^0 = [x_1^{10}, x_2^{20}, .., x_n^0]^T$
→ Lo más común: $X^0 = [0, 0, \ldots, 0]^T$

4 **Definir** el error máximo permitido (tolerancia) → ε

5 **Definir** el número máximo de iteraciones permitido → k_{max}

6 **Verificar** si A es diagonalmente dominante.

8 **Inicializar:** $k - 0$, $X - X^0$, $error - 1000$

9. **Mientras** ($error > \varepsilon$ y $k < k_{max}$) hacer:

9.1 Guardar $X_{anterior} \rightarrow X$

9.2 Para $i = 1$ hasta n:

$suma = 0$

Para $j = 1$ hasta n:

Si $j < i$:

$suma += a_{ij} * x_j$

Si $j > i$: $= a_{ij} * x_{j_anterior}$

$x_i = (b_i - suma) / a_{ii}$

9.3 **error** = máximo $[x_i - x_{i..anterior}]$

9.4 $k + k + 1$

10. Si $error \leq \epsilon$ entonces:

→ **Convergió:** $X = [x_1, x_2, .., x_n]$

Sino: → No convergió

Fin

$$\begin{bmatrix} A & a_1 & A \\ A & a_2 & A \\ A & a_3 & A \end{bmatrix} + \left(x_i^{k+1} = \frac{b_i - s_{aij+x_j}}{a_{ii}} \right.$$

Problema de contexto que soluciona este código

En un **taller de reparación de computadoras** se ofrecen tres tipos de servicios:

- x= número de reparaciones de **hardware** realizadas en un día
- y= número de **instalaciones de software** realizadas
- z= número de **mantenimientos preventivos**

Al final del día se obtuvieron los siguientes registros relacionados con el trabajo realizado:

1. La suma de dos reparaciones de hardware, una instalación de software y un mantenimiento preventivo equivale a **8 unidades de trabajo**.
2. Tres reparaciones de hardware menos una instalación de software más dos mantenimientos equivalen a **11 unidades de trabajo negativo debido a ajustes administrativos**.
3. Dos reparaciones de hardware menos una instalación de software más dos mantenimientos equivalen a **3 unidades de trabajo**.

Esto genera el siguiente **sistema de ecuaciones lineales**:

$$\begin{aligned} 2x + y - z &= 8 \\ -3x - y + 2z &= -11 \\ -2x + y + 2z &= -3 \end{aligned}$$

Determinar cuántos servicios de cada tipo (x, y, z) se realizaron durante el día utilizando métodos numéricos como **Gauss-Jordan** o **Gauss-Seidel**.

Código en Python

```python
import numpy as np
import time

# Sistema de ecuaciones
A = np.array([
    [2, 1, -1],
    [-3, -1, 2],
    [-2, 1, 2]
], dtype=float)

b = np.array([8, -11, -3], dtype=float)


# Metodo Gauss-Jordan
def gauss_jordan(A, b):
    n = len(b)
    M = np.hstack((A, b.reshape(-1,1)))

    for i in range(n):

        # Hacer pivote = 1
        M[i] = M[i] / M[i,i]

        # Hacer ceros en columna
        for j in range(n):
            if i != j:
                factor = M[j,i]
                M[j] = M[j] - factor * M[i]

    return M[:, -1]


# Metodo Gauss-Seidel
def gauss_seidel(A, b, tol=1e-6, max_iter=100):

    n = len(b)
    x = np.zeros(n)

    for k in range(max_iter):
        x_old = x.copy()
```

```python
        for i in range(n):
            suma = 0
            for j in range(n):
                if j != i:
                    suma += A[i][j] * x[j]

            x[i] = (b[i] - suma) / A[i][i]

        error = np.linalg.norm(x - x_old)

        if error < tol:
            return x, k+1, True

    return x, max_iter, False


# Comparacion de tiempos

# Gauss-Jordan
inicio = time.perf_counter()
sol_gj = gauss_jordan(A, b)
tiempo_gj = time.perf_counter() - inicio


# Gauss-Seidel
inicio = time.perf_counter()
sol_gs, iteraciones, converge = gauss_seidel(A, b)
tiempo_gs = time.perf_counter() - inicio


# Resultados

print("=== GAUSS JORDAN ===")
print("Solucion:", sol_gj)
print("Tiempo:", tiempo_gj)

print("\n=== GAUSS SEIDEL ===")
print("Solucion:", sol_gs)
print("Iteraciones:", iteraciones)
print("Converge:", converge)
print("Tiempo:", tiempo_gs)
```

Resultados obtenidos

GAUSS JORDAN

Solucion: [2. 3. -1.]

Tiempo: 0.00022339995484799147

GAUSS SEIDEL

Solucion: [2.86390955 -0.64998084 1.68889997]

Iteraciones: 100

Converge: False

Tiempo: 0.001640800037421286

En este caso vemos que el algoritmo de gauss seidel nunca converge, ya que la condición que dice que debe existir una diagonal dominante no se cumple.

Solución del sistema utilizando symbolab

$$2x + y - z = 8,\ -3x - y + 2z = -11,\ -2x + y + 2z = -3$$

Solución

$$x = 2, y = 3, z = -1$$

Mostrar pasos

Conclusión

En conclusión, al comparar los métodos de Gauss-Jordan y Gauss-Seidel se observa una diferencia clara tanto en precisión como en comportamiento numérico. El método de

Gauss-Jordan proporcionó una solución exacta $[2,3,-1]$en un tiempo muy bajo, lo que demuestra su efectividad para resolver sistemas de ecuaciones lineales de forma directa y confiable.

Por otro lado, el método de Gauss-Seidel no logró converger después de 100 iteraciones, arrojando una solución aproximada incorrecta $[2.8639,-0.6499,1.6889]$y un mayor tiempo de ejecución. Esto evidencia que su desempeño depende fuertemente de ciertas condiciones del sistema, como la existencia de una matriz diagonalmente dominante.

En este caso específico, Gauss-Jordan resulta ser el método más adecuado, ya que garantiza una solución exacta sin depender de criterios de convergencia. Mientras tanto, Gauss-Seidel puede ser útil en otros contextos, especialmente en sistemas grandes, pero requiere que se cumplan condiciones específicas para asegurar resultados correctos.

Funcionalidad del código

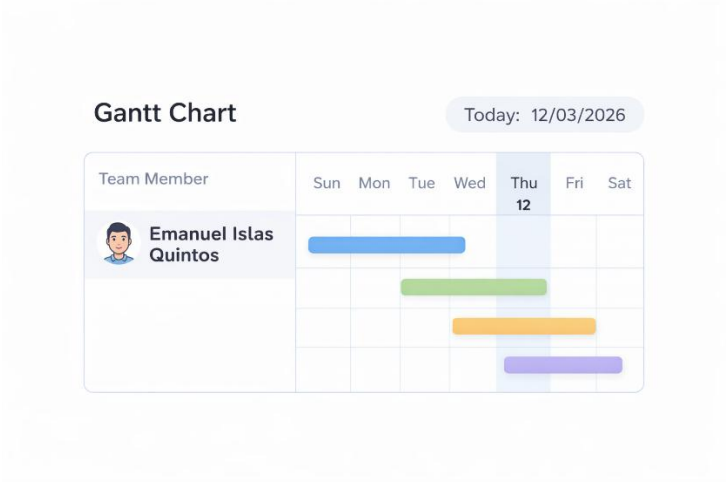
El programa desarrollado permite resolver sistemas de ecuaciones lineales utilizando los métodos de **Gauss-Jordan** y **Gauss-Seidel** (Siempre y cuando el sistema de ecuaciones ingresado cumpla con las características necesarias para que los métodos funcionen). Una de sus principales ventajas es que permiten comparar ambos métodos, observando la solución obtenida, el número de iteraciones en Gauss-Seidel y el tiempo de ejecución de cada uno.

Limitaciones del código actual

El método de Gauss-Jordan no incluye pivoteo, por lo que puede fallar si aparece un pivote igual a cero. En el caso de Gauss-Seidel, la convergencia no está garantizada para todos los sistemas de ecuaciones, especialmente si la matriz no es diagonalmente dominante.

Anexos

Roles y responsabilidades de los integrantes del equipo:

Integrantes	Responsabilidad
Emanuel Islas Quintos	todo

Grafica tipo Gantt